

Lesson 5 Worksheet: A Reusable Manhattan Function – ANSWER KEY

Part 1: Code Tracing – `compute_manhattan_path()` Step by Step

Trace the call: `compute_manhattan_path((0, 0), (2, 3))`

Setup variables:

Variable	Initial Value
<code>position</code>	<code>(0, 0)</code>
<code>current_row</code>	<code>0</code>
<code>current_col</code>	<code>0</code>
<code>dest_row</code>	<code>2</code>
<code>dest_col</code>	<code>3</code>
<code>path</code> (initial)	<code>[]</code>

South while loop – trace each iteration:

Iteration	<code>current_row</code> (before)	<code>< dest_row?</code>	<code>current_row</code> (after)	Appended to path
1	<code>0</code>	YES	<code>1</code>	<code>(1, 0)</code>
2	<code>1</code>	YES	<code>2</code>	<code>(2, 0)</code>
Check	<code>2</code>	NO	– (loop ends)	–

North while loop: Does it execute? NO Why? `current_row` (2) is not greater than `dest_row` (2), so the condition is false.

East while loop – trace each iteration:

Iteration	<code>current_col</code> (before)	<code>< dest_col?</code>	<code>current_col</code> (after)	Appended to path
1	<code>0</code>	YES	<code>1</code>	<code>(2, 1)</code>
2	<code>1</code>	YES	<code>2</code>	<code>(2, 2)</code>
3	<code>2</code>	YES	<code>3</code>	<code>(2, 3)</code>
Check	<code>3</code>	NO	– (loop ends)	–

West while loop: Does it execute? NO Why? `current_col` (3) is not greater than `dest_col` (3), so the condition is false.

Final returned path: [(1, 0), (2, 0), (2, 1), (2, 2), (2, 3)]

len(path): 5

Part 2: Trace a Reverse Path

Trace the call: `compute_manhattan_path((3, 3), (1, 0))`

Setup variables:

Variable	Initial Value
<code>current_row</code>	3
<code>current_col</code>	3
<code>dest_row</code>	1
<code>dest_col</code>	0

Which two while loops execute? North and West

North while loop:

Iteration	<code>current_row</code> (before)	<code>current_row</code> (after)	Appended
1	3	2	(2, 3)
2	2	1	(1, 3)

West while loop:

Iteration	<code>current_col</code> (before)	<code>current_col</code> (after)	Appended
1	3	2	(1, 2)
2	2	1	(1, 1)
3	1	0	(1, 0)

Final returned path: [(2, 3), (1, 3), (1, 2), (1, 1), (1, 0)]

len(path): 5

Part 3: Fill in the Blanks

```
def compute_manhattan_path(position, destination):
    """Compute a Manhattan path from position to destination."""
    path = []

    current_row, current_col = position
    dest_row, dest_col = destination

    # Move south
    while current_row < dest_row:
        current_row = current_row + 1
        path.append((current_row, current_col))

    # Move north
    while current_row > dest_row:
        current_row = current_row - 1
        path.append((current_row, current_col))

    # Move east
    while current_col < dest_col:
        current_col = current_col + 1
        path.append((current_row, current_col))

    # Move west
    while current_col > dest_col:
        current_col = current_col - 1
        path.append((current_row, current_col))

    return path
```

Blanks in order:

1. position
2. destination
3. []
4. position
5. destination
6. <
7. 1
8. append
9. current_row
10. current_col
11. >

```
12. 1
13. append
14. current_row
15. current_col
16. <
17. 1
18. append
19. current_row
20. current_col
21. >
22. 1
23. append
24. current_row
25. current_col
26. path
```

Part 4: Predict the Output

Program A:

Line 1: [(0, 1), (0, 2), (0, 3)]

Line 2: Steps: 3

What kind of edge case is this? Same row – the south and north loops do not execute, so the path only has column movement (east).

Program B:

Line 1: []

Line 2: Steps: 0

Why is the path empty? position equals destination, so none of the four while loop conditions are true. No loops execute, and the path stays as the empty list [].

Program C:

Line 1: Path 1: [(1, 0), (2, 0), (2, 1), (2, 2), (2, 3)]

Line 2: Path 2: [(1, 0), (1, 1)]

Do both paths start from the same position? YES

Why? Both calls pass (0, 0) explicitly as the position argument – the function has no memory between calls, so every call starts fresh from whatever position you hand it.

Part 5: Find and Fix the Bug

Bug 1:

What is wrong? `path` starts as `[position]` instead of `[]`. The starting position should NOT be in the path (the robot is already there). This would add an extra element at the beginning, and `len(path)` would be one more than the actual number of steps.

Fix: Change `path = [position]` to `path = []`

Bug 2:

What is wrong? The west loop adds 1 instead of subtracting 1: `current_col = current_col + 1`. This makes the column go the wrong direction (east instead of west), and the loop condition `current_col > dest_col` stays true forever, creating an infinite loop.

What would happen if you called `compute_manhattan_path((3, 3), (2, 3))`? The north loop runs correctly, moving row from 3 to 2. Then the west loop should move column from 3 to 2 – wait, there's no column movement needed here since both columns are 3. Only the north loop would run, so this particular bug wouldn't trigger. It would trigger on any call where the destination's column is less than the starting column, e.g. `compute_manhattan_path((3, 3), (2, 1))` – the west loop should move column from 3 to 1, but instead it adds 1 each time (3, 4, 5, 6, ...), creating an infinite loop that never terminates.

Fix: Change `current_col = current_col + 1` to `current_col = current_col - 1` in the west while loop.

Bug 3:

What is wrong? In the east while loop, the append call has row and column swapped: `path.append((current_col, current_row))` instead of `path.append((current_row, current_col))`. This puts the column value in the row position and vice versa.

If position is (0, 0) and destination is (2, 3), what would the INCORRECT last three elements of the path be?

(1, 2), (2, 2), (3, 2)

What should they be?

(2, 1), (2, 2), (2, 3)

Reflection

`compute_manhattan_path()` uses 4 separate while loops instead of if/else statements with 2 while loops. Why does this simpler approach work? (Hint: think about what happens when a while loop's condition is already false.)

A while loop whose condition is already false simply does nothing – it is skipped entirely. So we can write all four directions (south, north, east, west) as separate loops without any if/else logic. For any given path, only the one or two loops whose conditions are true will actually execute. For example, going from (0,0) to (2,3), only the south loop (`current_row < dest_row`) and east loop (`current_col < dest_col`) run. The north and west loops are automatically skipped because their conditions are false. This is simpler than using if/else to choose between +1 and -1 step values.

Part 6 (Optional Extension): Wrap It in a Class

1. Fill in the blanks:

```
class Manhattan:
    def __init__(self, start):
        self.position = start

    def compute_path(self, destination):
        path = []
        current_row, current_col = self.position
        ...
```

2. Rewrite the two calls using the class version:

```
nav = Manhattan((0, 0))
path1 = nav.compute_path((2, 3))
path2 = nav.compute_path((1, 1))
```

3. Does `compute_path` (the method) ever change `self.position`?

NO - Why or why not? It only reads `self.position` when unpacking into `current_row` / `current_col` - it never reassigns `self.position` itself. If it did, the object's stored starting position would drift after every call, and a second `compute_path()` call would start from the wrong place.